PROGRAMMING & CS FUNDAMENTALS

Heaps & Priority Queues Visual Guide

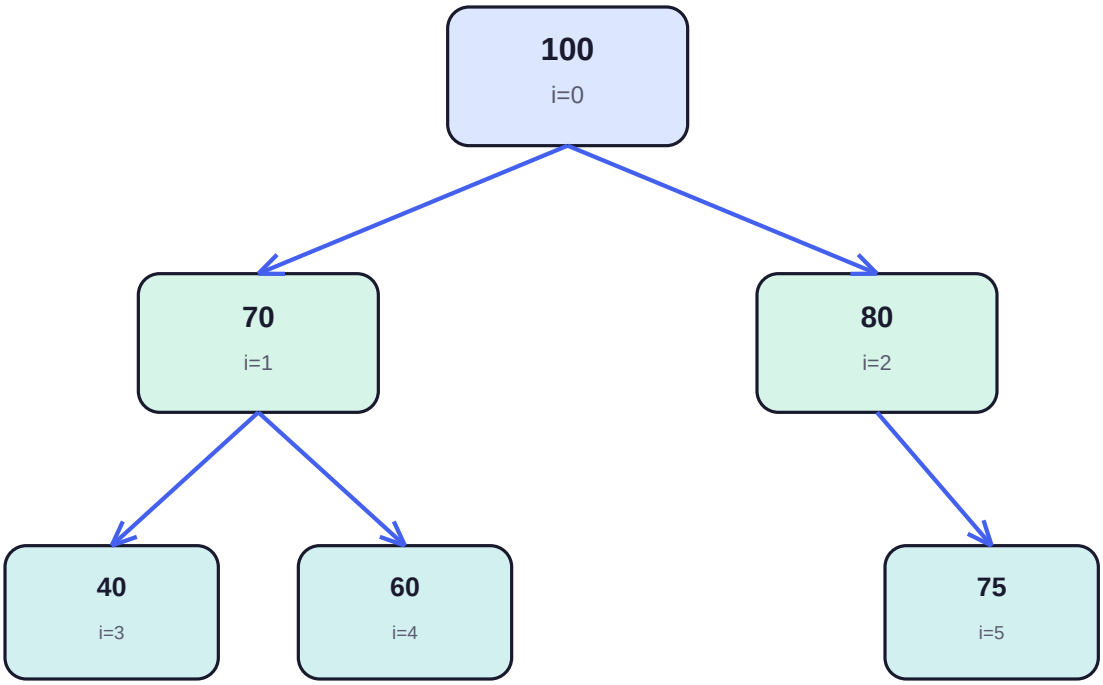
'Most urgent first' —
the structure behind schedulers and top-k.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

The Heap Property + Array Embedding

Parent beats children; the tree lives inside a flat array.



`array: [100, 70, 80, 40, 60, 75]`

```
parent(i) = (i-1)//2;  left(i) = 2i+1;  right(i) = 2i+2
max-heap: parent >= children   # root = max, always
NOT sorted: [100,70,80,...] — only the heap order holds
```

Complete binary tree: filled left-to-right, so the array has no holes. A heap is a priority queue — NOT a sorted list.

STEP 2/6

Sift Up, Sift Down, Heapify

All heap ops are 'restore the property by swapping along one path'.

```
push(x): append at end, SIFT UP while x > parent     # O(log n)
pop():   take root, move last -> root, SIFT DOWN     # O(log n)
peek():  root                                        # O(1)
heapify(arr): sift down from last parent -> root     # O(n)!
```

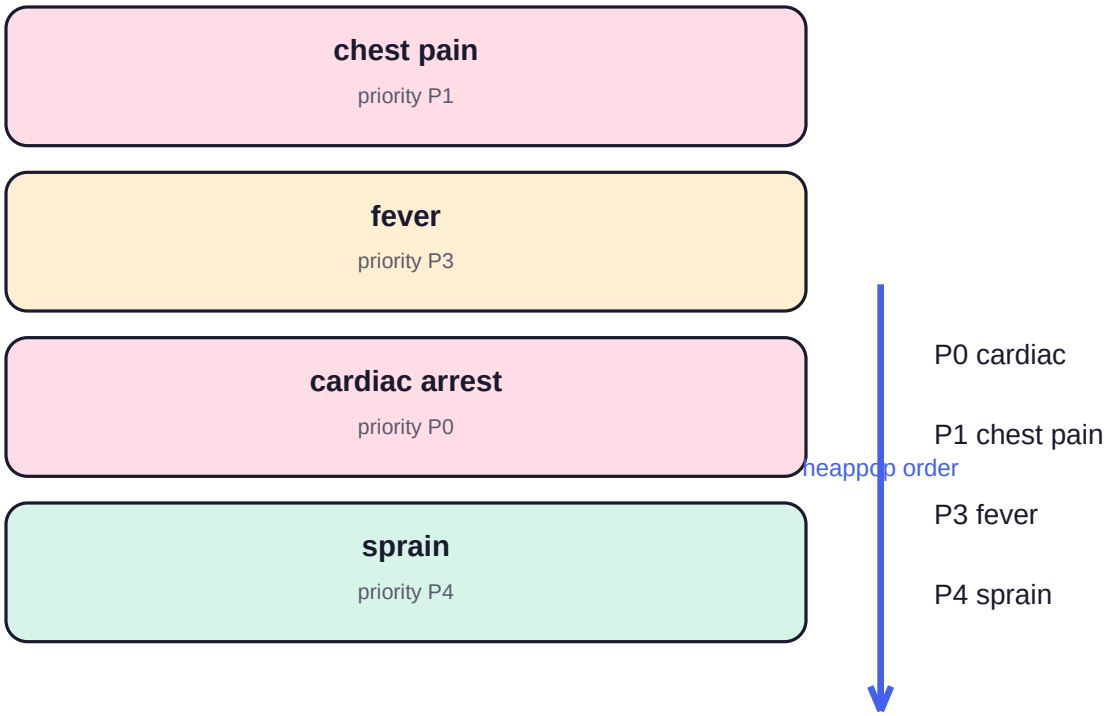
- Sift up/down each walk one root-to-leaf path: height = log n.
- heapify is O(n), not O(n log n): most nodes sit near the leaves (short sifts).
- Building by n pushes costs O(n log n) — heapify the array instead.
- Python: heapq is a MIN-heap; negate values for max-heap behavior.

Interview trap: 'sort with a heap' = heapsort O(n log n). 'K smallest' = heap of size k (next step).

STEP 3/6

Priority Queue in Action

ER triage: severity outranks arrival order.



- OS schedulers, Dijkstra's frontier, A* search, event simulation — all priority queues.
- Stable tie-break: push (priority, counter, item) so FIFO breaks ties deterministically.
- Decrease-key: Dijkstra needs it; lazy deletion (skip stale entries) avoids the complexity.

STEP 4/6

Pattern: Top-K

K largest in a stream — keep a min-heap of exactly k.

```
import heapq
h = []
for x in stream:                        # n items, unknown length
    heapq.heappush(h, x)
    if len(h) > k: heapq.heappop(h)    # evict the smallest
# h holds the k largest; root = kth largest.  O(n log k)
```

stream: 5, 1, 9, 3, 7, ... k=3 -> heap keeps [5, 7, 9]

- O(n log k) time, O(k) space — beats full sort O(n log n) when k is much smaller than n.
- Mirror: k smallest -> max-heap of size k (negate in Python).
- Median of a stream: max-heap (low half) + min-heap (high half) — the two-heap tango.

'Top k frequent': count with a hash map, then top-k the counts — hash table + heap, the classic combo.

STEP 5/6

Scheduling with Heaps

Merge k sorted lists; schedule tasks with cooldowns.

```
# merge k sorted lists: heap of (head_value, list_id)
push each list's head; repeatedly pop min, push its successor
# => O(N log k) for N total elements

# task scheduler (cooldown n): max-heap of remaining counts
# + queue of cooling tasks; always run the most-remaining task
```

- K-way merge beats pairwise merging: O(N log k) vs O(N k).
- Huffman coding: repeatedly merge two lightest nodes via min-heap — optimal prefix codes.
- CPU scheduling (priority), bandwidth allocation, discrete-event sims: heaps schedule the future.

STEP 6/6

Heap Facts & Traps

What the heap is not.

- A heap is NOT sorted — only root-vs-children order holds. Don't binary-search it.
- Find-min in a max-heap is O(n): the min hides among the leaves.
- Heap vs sorted array: heap wins for repeated extract-min; array wins for scans.
- d-ary heaps (more children): shallower, better cache behavior — used in real Dijkstra.
- Binomial/Fibonacci heaps: theoretically faster decrease-key; rarely worth the complexity.
- Memory heaps (malloc arenas) share only the NAME — different concept entirely.

STEP ✓I✓

Key Takeaways

Heaps & Priority Queues — Remember

- Heap property (parent beats children) in a complete tree embedded in an array:

  parent(i)=(i-1)//2.
- Push/pop = sift along one path: O(log n); peek O(1); heapify an array in O(n).
- Priority queues power triage, schedulers, Dijkstra, A*, and event simulation.
- Top-k pattern: bounded heap of size k -> O(n log k) time, O(k) space.
- A heap is not sorted — never binary-search it; min of a max-heap is O(n).

Keep exploring

Pair this guide with the matching video walkthroughs
and hands-on labs at interstitiumlabs.dev